



ADR-003: JSON Logic rules engine with maker-checker governance

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

- **Status:** Accepted
- **Date:** 2026-10-07
- **Related:** [ADR-008](#), [ADR-009](#), [AI governance](#)

Context and problem statement

Much of the platform's behaviour changes far more often than code is released, and much of it is regulated:

- tariffs, commission and statutory caps;
- scoring weights, journeys and next-best-actions;
- message templates, the voice bot script and banned phrases;
- consent and contact policy, ABAC and retention.

Business owners (product, campaign, compliance, actuarial, DPO) need to change these values **safely**. That means validation before saving, four-eyes approval, a full audit trail, simulation and rollback, and no code deployment.

Decision drivers

- No code execution from data: no `eval`, no `Function`, no prototype access.
- An invalid rule must be impossible to activate.
- Segregation of duties between author and approver.
- Every score and NBA must be explainable (rule id and reason).
- Replicas must converge quickly after an approval.

Considered options

1. **Safe JSON Logic subset + decision tables + per-kind validators + maker-checker workflow in the platform.**
2. An external BRMS (Drools, Camunda DMN, FICO Blaze).
3. Rules as code (feature flags + releases).
4. Full JSON Logic library from npm.

Decision outcome

Chosen option: **1**.

- **Evaluator** (`src/rules/jsonLogic.js`). Operator allow-list (`var`, `missing`, `==`, `...`, `%`, `min`, `max`, `clamp`, `round`, `in`, `cat`, `and`, `or`, `if`). `and`, `or` and `if` evaluate lazily. `__proto__`, `prototype` and `constructor` are forbidden both as operators and as `var` path segments. `getVar` uses `hasOwnProperty`, so it never walks the prototype chain. Division by zero returns 0.
- **Decision tables** (`src/rules/decisionTable.js`). `hitPolicy` is `first` or `collect`. Rows need unique ids. `{{path}}` templates are allowed in outputs. Each result carries a `ruleId` for explainability.
- **Validators** (`src/rules/validators.js`). Every embedded expression and table is validated, plus kind-specific checks:
 - products: code format, known rating method, bundle references;
 - tariff: VAT between 0 and 0.2, positive integer premiums;
 - scoring: weights sum to 100, hot > warm;
 - journeys: valid anchor, channels present;

- contact policy: window sanity;
- commission: $\text{rate} \leq \text{statutory cap}$;
- **copy guard** on every customer-facing string of `content.*` and `benefits`.
- **Lifecycle** (`src/application/rulesService.js`): `draft` → `pending_approval` → `active` → `retired`, or → `rejected`.
 - Only the author can submit.
 - **The approver must not be the author** (enforced in `approve` and `reject`).
 - Approval retires the previous active version (`supersededBy`), records the approver and comment, writes `rules.approved` to the audit chain and publishes `rules.activated`.
 - Rollback creates a **new draft** from an earlier version, so it is approved again.
 - Each version stores a SHA-256 checksum (first 16 hex characters).
- **Cache.** Active rule sets are cached in-process with a 15 s TTL. `rules.activated` invalidates the cache on the replica that handles the event, so every replica converges within 15 s.
- **Simulation.** `POST /api/rules/simulate` compares current and candidate scoring, NBA, journeys or benefits for a real profile before submission.
- **Bootstrap.** `config/rules/*.json` seeds version 1 of each kind (as `system`) only when the kind has no versions yet. `copy_guard` loads first so content is guarded from the first seed.

Consequences

- Good: business change lead time is hours, not a release cycle ($\text{target} \leq 1 \text{ day}$, `insightsService.adoption`).
- Good: rules are data, so they can be diffed (checksums), audited and exported for regulators.
- Good: a small, auditable evaluator with no npm dependency.
- Bad: JSON Logic is verbose for authors. **Mitigation:** the rules console with validation and simulation, plus templates per kind.
- Bad / gaps to close:
 - **Activation is not atomic.** Retire-previous and activate-new are separate writes and not in `store.transaction`. Two concurrent approvals of different versions of the same kind could leave two `active` rows. The cache then picks whichever row loads last. **Fix:** wrap `approve` in a transaction and add a partial unique index `CREATE UNIQUE INDEX ux_rulesets_active ON rulesets(kind) WHERE status='active'`.
 - **The commission cap check matches by string search** on the serialised `when` (`validators.js`). A row that applies to all products (no `when`) and the table `default` are not checked against caps. Runtime `commissionFor` still clamps with `Math.min(rate, cap)` and returns `capped: true`, so payouts cannot exceed the cap. Even so, the validator should evaluate each row against each capped product.
 - **No expression size or depth limit.** The HTTP body limit (1 MiB) bounds it today. **Fix:** add a node-count limit in `validate`.
 - **No role binding per kind.** Any `rules:approve` holder can approve any kind. **Fix:** add an `approverRoles` map per kind, for example `abac` → InfoSec and `tariff.*` → compliance.

Pros and cons of the options

External BRMS. Powerful authoring and DMN standards. However, it means licence cost, another runtime to secure and operate, and integration latency. It is overkill for the current rule volume.

Rules as code. Strong typing and review. However, every tariff or template change becomes a release, and business owners cannot self-serve.

Full JSON Logic library. More operators. However, it adds a dependency and supports operators we do not want, such as method calls on data in some implementations.